

NAME: - A. MONESH

REG NO: - 192311430

Q1. Apriori Algorithm – Online Shopping Transactions

The dataset contains the following transactions:

Order ID Products Purchased

O1	Mobile, Earphones
O2	Mobile, Charger
O3	Earphones, Charger
O4	Mobile, Earphones, Charger
O5	Mobile, Earphones

Question: Design pseudocode for the Apriori algorithm to discover frequent product combinations using a specified minimum support threshold, and illustrate candidate generation and pruning at each iteration.

Pseudocode

ALGORITHM Apriori_Online_Shopping

INPUT:

```
Transactions = {  
    O1: {Mobile, Earphones},  
    O2: {Mobile, Charger},  
    O3: {Earphones, Charger},  
    O4: {Mobile, Earphones, Charger},  
    O5: {Mobile, Earphones}  
}
```

Minimum_Support = specified threshold

OUTPUT:

All frequent itemsets

BEGIN

1. Find all individual products in the transactions.

2. Generate C1:

C1 = all 1-item candidate itemsets.

3. Count the support of every itemset in C1.

4. Prune C1:

Remove itemsets whose support is less than

Minimum_Support.

5. Store the remaining itemsets as L1.

6. Set $k = 2$.

7. WHILE $L(k-1)$ is not empty DO

a. Generate candidate itemsets C_k

by joining itemsets in $L(k-1)$.

b. FOR each candidate itemset c in C_k DO

i. Check all $(k-1)$ -subsets of c .

ii. IF any subset is not present in $L(k-1)$

THEN

Remove c from C_k .

END IF

END FOR

c. Count the support of every remaining
candidate in C_k .

d. Prune candidates:

Remove candidates whose support
is less than Minimum_Support.

e. Store the remaining itemsets as L_k .

f. Increment k by 1.

END WHILE

8. Combine $L_1, L_2, L_3, \dots$ to obtain
all frequent itemsets.

9. Display all frequent itemsets.

END

Candidate Generation and Pruning

Iteration 1 – 1-itemsets

Candidate set:

$C_1 = \{\text{Mobile}\}, \{\text{Earphones}\}, \{\text{Charger}\}$

Support counts:

Mobile = 4

Earphones = 4

Charger = 3

After applying the minimum support threshold, itemsets satisfying the threshold are retained in **L_1** .

Iteration 2 – 2-itemsets

Candidates are generated by joining frequent 1-itemsets:

$C_2 =$

$\{\text{Mobile}, \text{Earphones}\}$

$\{\text{Mobile}, \text{Charger}\}$

$\{\text{Earphones}, \text{Charger}\}$

Their support counts are:

$\{\text{Mobile, Earphones}\} = 3$

$\{\text{Mobile, Charger}\} = 2$

$\{\text{Earphones, Charger}\} = 2$

Candidates whose support is below the minimum support threshold are **pruned**.

Iteration 3 – 3-itemsets

The remaining frequent 2-itemsets are joined to generate:

$C3 = \{\text{Mobile, Earphones, Charger}\}$

Its support is:

$\{\text{Mobile, Earphones, Charger}\} = 1$

If this support is below the minimum support threshold, the candidate is **pruned**.

Apriori Process

Transactions



Generate C1



Count Support



Prune Infrequent Itemsets



L1



Generate C2



Count Support



Prune



L2



Generate C3



Count Support



Prune



Frequent Itemsets

Key Apriori principle:

If an itemset is infrequent, all of its supersets will also be infrequent. Therefore, they can be pruned without further support counting.

This answer directly follows the question's requirement for **candidate generation, support counting, and pruning at each iteration**.

Q2. FP-Growth Algorithm – Library Book Borrowing Records

The dataset contains these borrowing records:

Borrow ID Books Borrowed

B1	Data Science, Python
B2	Python, Machine Learning
B3	Data Science, Machine Learning
B4	Data Science, Python, Machine Learning
B5	Python, Machine Learning

Question: Develop pseudocode for the **FP-Growth algorithm** to identify frequent borrowing patterns **without generating candidate itemsets**. Explain the steps involved in constructing the FP-Tree using the given records.

Pseudocode

ALGORITHM FP_Growth(Transactions, Minimum_Support)

INPUT:

Transactions = borrowing records

Minimum_Support = specified support threshold

OUTPUT:

Frequent itemsets

BEGIN

1. Scan all transactions and count the support

of each individual book.

2. Remove books whose support is less than
Minimum_Support.
3. Sort the remaining books in descending order
of their support count.
4. Create an empty FP-Tree with a root node.
5. FOR each transaction DO
 - a. Remove infrequent books.
 - b. Arrange the remaining books according to
the global descending support order.
 - c. Insert the ordered books into the FP-Tree.
 - d. If a node already exists, increment its
count.
 - e. Otherwise, create a new node.
 - f. Update the node links in the header table.

END FOR

6. Apply FP-Growth recursively:
 - a. Select an item from the header table.
 - b. Find its conditional pattern base.
 - c. Construct its conditional FP-Tree.
 - d. Generate frequent patterns from the
conditional FP-Tree.

- e. Combine the selected item with the generated patterns.
 - 7. Repeat until the conditional FP-Tree becomes empty or contains only one path.
 - 8. Output all discovered frequent itemsets.
- END

Construction of FP-Tree

First, count the occurrence of each book.

Book	Support Count
Python	4
Machine Learning	4
Data Science	3

The books are arranged according to descending support. One possible order is:

Python → Machine Learning → Data Science

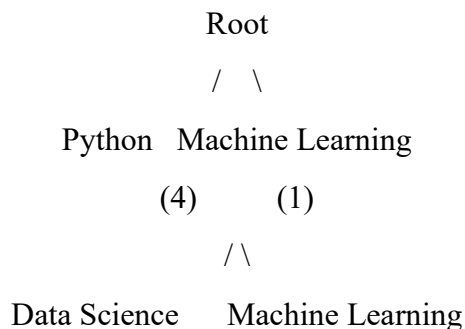
Now reorder each transaction according to this order:

Borrow ID Ordered Transaction

B1	Python → Data Science
B2	Python → Machine Learning
B3	Machine Learning → Data Science
B4	Python → Machine Learning → Data Science
B5	Python → Machine Learning

FP-Tree Insertion

The transactions are inserted one by one.



(1) (3)

/

Data Science

(1)

Common prefixes are **shared** in the FP-Tree. For example, B2, B4, and B5 begin with Python → Machine Learning, so the same path can be shared and its counts are increased rather than creating separate candidate itemsets.

FP-Growth Mining Process

Library Transactions



Count Item Frequencies



Remove Infrequent Items



Sort Items by Support



Construct FP-Tree



Create Header Table



Find Conditional Pattern Base



Construct Conditional FP-Tree



Mine Frequent Patterns



Frequent Borrowing Patterns

Important Point

Unlike **Apriori**, FP-Growth **does not generate candidate itemsets**. Instead, it:

1. Compresses the transaction database into an **FP-Tree**.
2. Uses the **header table** to access items.

3. Builds **conditional pattern bases**.
4. Creates **conditional FP-Trees**.
5. Recursively mines frequent patterns.

This directly addresses the question's requirement to identify borrowing patterns **without candidate generation** and to explain FP-Tree construction.

Q3. Generate Association Rules from Frequent Service Combinations

The given frequent service combinations and support counts are:

Service Combination	Support Count
{Internet}	4
{Streaming}	4
{Cloud Storage}	3
{Internet, Streaming}	3
{Streaming, Cloud Storage}	2

The task is to generate association rules by calculating **confidence**, generating rules, and filtering them using a **minimum confidence threshold**.

Pseudocode

ALGORITHM Generate_Association_Rules

INPUT:

Frequent_Itemsets

Minimum_Confidence

OUTPUT:

Strong Association Rules

BEGIN

1. FOR each frequent itemset F in Frequent_Itemsets DO
2. IF number of items in F ≥ 2 THEN
3. Generate all non-empty proper subsets

of F.

4. FOR each subset A of F DO
5. $B = F - A$
6. Calculate confidence:

 $\text{Confidence}(A \rightarrow B)$
 $= \text{Support}(F) / \text{Support}(A)$
7. IF $\text{Confidence}(A \rightarrow B)$
 $\geq \text{Minimum_Confidence}$ THEN
8. Generate rule:
 $A \rightarrow B$
9. Store the rule and its
 confidence value.
10. END IF
11. END FOR
12. END IF
13. END FOR
14. Display all strong association rules.

END

Step-by-Step Rule Generation

Consider the frequent itemset:

{Internet, Streaming}

Support = 3

Possible rules are:

Rule 1:

Internet $\rightarrow$ Streaming

Confidence:

$$\begin{aligned} & \text{Confidence}(\text{Internet} \rightarrow \text{Streaming}) \\ &= \text{Support}(\text{Internet, Streaming}) / \text{Support}(\text{Internet}) \end{aligned}$$

$$= 3 / 4$$

$$= 0.75 = 75\%$$

Rule 2:

Streaming $\rightarrow$ Internet

Confidence:

$$\begin{aligned} & \text{Confidence}(\text{Streaming} \rightarrow \text{Internet}) \\ &= \text{Support}(\text{Internet, Streaming}) / \text{Support}(\text{Streaming}) \end{aligned}$$

$$= 3 / 4$$

$$= 0.75 = 75\%$$

If the **minimum confidence is 70%**, both rules are retained.

Another Example

For the frequent itemset:

{Streaming, Cloud Storage}

Support = 2

Possible rules:

Streaming $\rightarrow$ Cloud Storage

Cloud Storage $\rightarrow$ Streaming

For:

Streaming $\rightarrow$ Cloud Storage

$$\begin{aligned} & \text{Confidence} \\ &= \text{Support}(\text{Streaming, Cloud Storage}) / \text{Support}(\text{Streaming}) \end{aligned}$$

$$= 2 / 4$$

$$= 50\%$$

If the minimum confidence is **70%**, this rule is **filtered out**.

For:

Cloud Storage $\rightarrow$ Streaming

Confidence

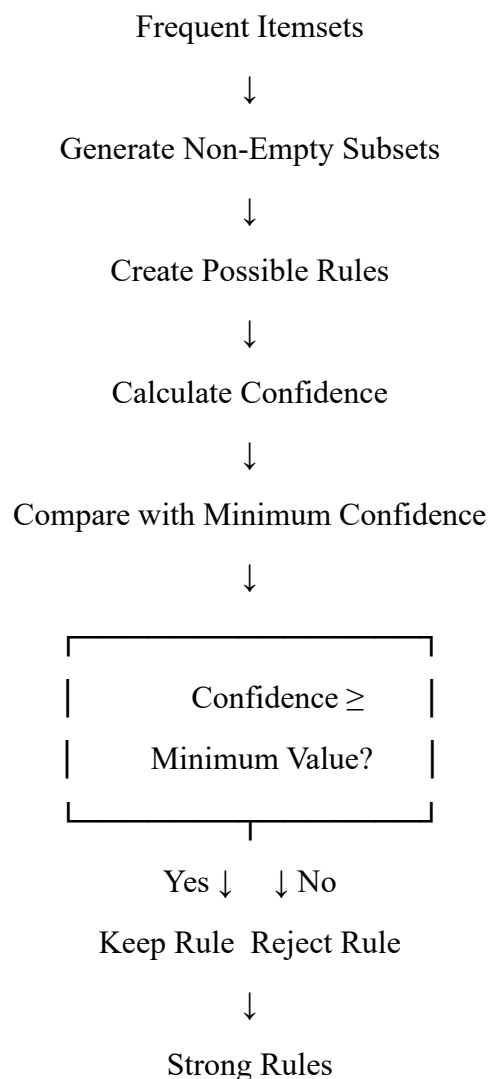
$$= \text{Support}(\text{Streaming, Cloud Storage}) / \text{Support}(\text{Cloud Storage})$$

$$= 2 / 3$$

$$= 66.67\%$$

This is also **filtered out** when minimum confidence is 70%.

Overall Process



Important Formula

Confidence measures how often the items in the consequent occur when the antecedent occurs:

$$\text{Confidence}(A \rightarrow B)$$

$$= \text{Support}(A \cup B) / \text{Support}(A)$$

Thus, the algorithm generates possible rules from the given frequent combinations, calculates their confidence, and retains only those satisfying the specified minimum confidence threshold, exactly as required in Q3.

Q4. Constraint-Based Frequent Itemset Mining

The dataset contains the following student course enrollments:

Student ID Courses Enrolled

S1	Python, Database
S2	Python, AI
S3	Database, AI
S4	Python, Database, AI
S5	Python

Constraint: Only itemsets **containing the course "Python"** should be considered during mining.

Pseudocode

ALGORITHM Constraint_Based_Frequent_Itemset_Mining

INPUT:

Transactions = Student course enrollments

Minimum_Support = specified threshold

Constraint = Itemset must contain "Python"

OUTPUT:

Frequent itemsets containing Python

BEGIN

1. Scan all transactions.
2. Select "Python" and find its support count.
3. IF support(Python) >= Minimum_Support THEN
 Add {Python} to L1.

ELSE

STOP.

END IF

4. Generate candidate 2-itemsets only by combining
{Python} with other courses.
5. FOR each candidate itemset C DO

- a. Check whether C contains "Python".

- b. IF C does not contain "Python" THEN

- Prune C.

- ELSE

- Count its support.

- END IF

END FOR

6. Remove candidates whose support is less than
Minimum_Support.
7. Store the remaining candidates as frequent
itemsets.
8. FOR $k = 3$ onwards DO

- a. Generate candidates by joining the
frequent $(k-1)$ -itemsets.

- b. Generate only candidates containing
"Python".

- c. Check all required subsets.

d. Prune candidates that:

- do not contain Python, OR
- have support below Minimum_Support.

e. Store the remaining itemsets as Lk.

END FOR

9. Continue until no new frequent itemsets

can be generated.

10. Display all frequent itemsets containing Python.

END

Candidate Generation

From the given dataset, the individual courses are:

Python

Database

AI

Since every candidate **must contain Python**, candidates such as {Database, AI} are immediately excluded.

Therefore, instead of generating all possible combinations:

{Python, Database}

{Python, AI}

{Database, AI}

we generate only:

{Python, Database}

{Python, AI}

This reduces unnecessary candidate generation.

Support Counting

For the given records:

$\text{Support}(\{\text{Python}\}) = 4$

$\text{Support}(\{\text{Python}, \text{Database}\}) = 2$

$\text{Support}(\{\text{Python}, \text{AI}\}) = 2$

The itemset $\{\text{Python}, \text{Database}, \text{AI}\}$ occurs only in S4:

$\text{Support}(\{\text{Python}, \text{Database}, \text{AI}\}) = 1$

Each itemset is retained or pruned according to the specified minimum support threshold.

Effect of the Constraint

The constraint is:

Every frequent itemset **MUST** contain "Python"

Therefore:

- $\{\text{Python}\} \rightarrow \textbf{considered}$
- $\{\text{Python}, \text{Database}\} \rightarrow \textbf{considered}$
- $\{\text{Python}, \text{AI}\} \rightarrow \textbf{considered}$
- $\{\text{Python}, \text{Database}, \text{AI}\} \rightarrow \textbf{considered}$
- $\{\text{Database}, \text{AI}\} \rightarrow \textbf{pruned immediately}$

Process

Student Transactions



Apply Python Constraint



Generate Only Python-Based Candidates



Count Support



Prune Low-Support Itemsets



Generate Larger Python-Based Itemsets



Repeat Until No More Candidates



Frequent Itemsets Containing Python

Conclusion

Constraint-based mining improves efficiency because the **Python constraint is applied during candidate generation itself**. Itemsets that cannot satisfy the constraint are discarded early, reducing the number of candidates that need support counting. This directly addresses the question's requirement to explain how the constraint affects **candidate generation and pruning**.

Q5. Multilevel Association Rule Mining

The dataset contains a **treatment hierarchy** for five patients.

Patient ID Treatments Received

P1	Healthcare, Cardiology, ECG
P2	Healthcare, Orthopedics
P3	Healthcare, Cardiology
P4	Healthcare, Cardiology, ECG
P5	Healthcare, Orthopedics

The question asks for pseudocode to perform **multilevel association rule mining**, generate rules at different hierarchy levels, and explain how concept hierarchies improve pattern discovery.

Pseudocode

ALGORITHM Multilevel_Association_Rule_Mining

INPUT:

Treatment Transactions

Treatment Hierarchy

Minimum_Support

Minimum_Confidence

OUTPUT:

Multilevel Association Rules

BEGIN

1. Start from the highest level of the treatment hierarchy.
2. Generate itemsets at the current hierarchy level.
3. Calculate the support count of each itemset.
4. Remove itemsets whose support is less than Minimum_Support.
5. FOR each frequent itemset DO
 - a. Generate possible association rules.
 - b. Calculate confidence for each rule.
 - c. IF confidence $\geq$ Minimum_Confidence THEN
Store the rule.
END IF
- END FOR
6. Move to the next lower level of the treatment hierarchy.
7. Generate more specific treatment itemsets using the frequent patterns obtained from the previous level.
8. Calculate support for the new itemsets.

9. Prune itemsets that do not satisfy the Minimum_Support.
10. Generate association rules from the remaining frequent itemsets.
11. Calculate confidence for each rule.
12. Retain rules satisfying Minimum_Confidence.
13. Repeat Steps 6–12 until the lowest level of the hierarchy is reached.
14. Combine rules from all hierarchy levels.
15. Display the multilevel association rules.

END

Hierarchy Levels

The treatment data can be viewed at different levels of specificity:

Level 1: Healthcare



Level 2: Cardiology Orthopedics



Level 3: ECG

Thus, mining can begin with the **general treatment category** and then move toward more **specific treatments**.

Step-by-Step Mining

Level 1 – General Level

At the highest level, the common treatment is:

Healthcare

It occurs for all five patients:

$\text{Support}(\text{Healthcare}) = 5$

So it is a frequent item.

Level 2 – Specific Treatments

At the next level:

Cardiology = 3

Orthopedics = 2

Therefore, depending on the minimum support threshold, these treatment patterns can be retained or pruned.

Level 3 – More Specific Treatment

At the lower level:

ECG = 2

The algorithm checks its support and generates rules if the support and confidence thresholds are satisfied.

Rule Generation

For example, a possible rule involving treatments can be represented as:

Cardiology $\rightarrow$ ECG

The confidence is calculated as:

$\text{Confidence}(\text{Cardiology} \rightarrow \text{ECG})$

=

$\frac{\text{Support}(\text{Cardiology}, \text{ECG})}{\text{Support}(\text{Cardiology})}$

$\text{Support}(\text{Cardiology})$

For the given records:

$\text{Support}(\text{Cardiology}, \text{ECG}) = 2$

$\text{Support}(\text{Cardiology}) = 3$

$\text{Confidence} = 2 / 3$

= 66.67%

The rule is retained only if **66.67%** meets or exceeds the specified minimum confidence.

Overall Process

Treatment Transactions



Build Treatment Hierarchy



Mine General-Level Patterns



Calculate Support



Prune Infrequent Patterns



Move to More Specific Level



Generate Specific Patterns



Calculate Confidence



Filter Strong Rules



Multilevel Association Rules

How Concept Hierarchies Improve Pattern Discovery

A concept hierarchy allows the algorithm to discover patterns at **different levels of detail**:

- **Higher level:** discovers broad patterns such as Healthcare.
- **Middle level:** discovers more specific patterns such as Cardiology and Orthopedics.
- **Lower level:** discovers detailed patterns such as ECG.
- It allows both **general and specific relationships** to be discovered.
- It helps reveal patterns that may not be obvious when only one level of treatment information is analyzed.

Therefore, **multilevel association rule mining provides a hierarchical view of relationships**, starting from general treatment categories and progressively moving toward more detailed treatment patterns, as required by Q5.